INSTITUTO FEDERAL
São Paulo

ITPESDD

ESTRUTURA DE DADOS

Aula 4 - Conceitos sobre Análise e Complexidade de Algoritmos

Prof. Dr. Carlos Henrique da Silva Santos

carlos.santos@ifsp.edu.br

Planejamento da Disciplina - 1/1

Aula	Conteúdo
1	Revisão de Programação
2	Ponteiros
3	Emum e Struct
4	Análise de complexidade
5	Exercícios
6	**Avaliação 1**
7	Lista ligada simples
8	Lista circular
9	Pilha

Planejamento da Disciplina - 2/3

Aula	Conteúdo
10	Fila
11	Exercícios
12	**Avaliação 2**
13	Ordenação BubbleSort e MergeSort
14	Quicksort
15	Árvore Binária
16	Árvore Binária
17	Exercícios
18	**Avaliação 3**
19	Revisão da Disciplina

Sugestões de Fontes de Estudos

UNIVESP
Universidade Virtual do Estado de São Paulo

Estrutura de Dados - UNIVESP

Massachusetts Institute of Technology

Introduction to Algorithms

Castilho, Marcos Alexandre et al. **Algoritmos e estruturas de dados 1** [recurso eletrônico] / - Curitiba, 2020.
Disponível em: https://www.inf.ufpr.br/marcos/livro_alg1/livro_alg1.pdf
Acessado em: 14/10/2025

Toscani, L. V., Veloso, P. A. S., Complexidade de algoritmos: análise, projeto e métodos, 3 ed. Bookman, 2012

Algoritmos

Um algoritmo resolve um problema quando, para qualquer entrada, produz uma resposta correta, se forem concedidos tempo e memória suficientes para sua execução. O fato de um algoritmo resolver (teoricamente) um problema não significa que se seja aceitável na prática. Os recursos de espaço e tempo requeridos têm grande importância em casos práticos.

Toscani, L. V., Veloso, P. A. S., Complexidade de algoritmos: análise, projeto e métodos, 3 ed. Bookman, 2012

Algoritmos

Resolvendo um sistema linear 20x20

tabela 1.1.1 Tamanho do problema × tempo de execução

n	Método de Cramer	Método de Gauss
2	22 μs	50 μs
3	102 μs	150 μs
4	456 μs	353 μs
5	2,35 ms	666 μs
10	1,19 min.	4,95 ms
20	15225 séculos	38,63 ms
40	5.10^{33} séculos	0,315 s

Toscani, L. V., Veloso, P. A. S., Complexidade de algoritmos: análise, projeto e métodos, 3 ed. Bookman, 2012

Algoritmos

Complexidade de algoritmo

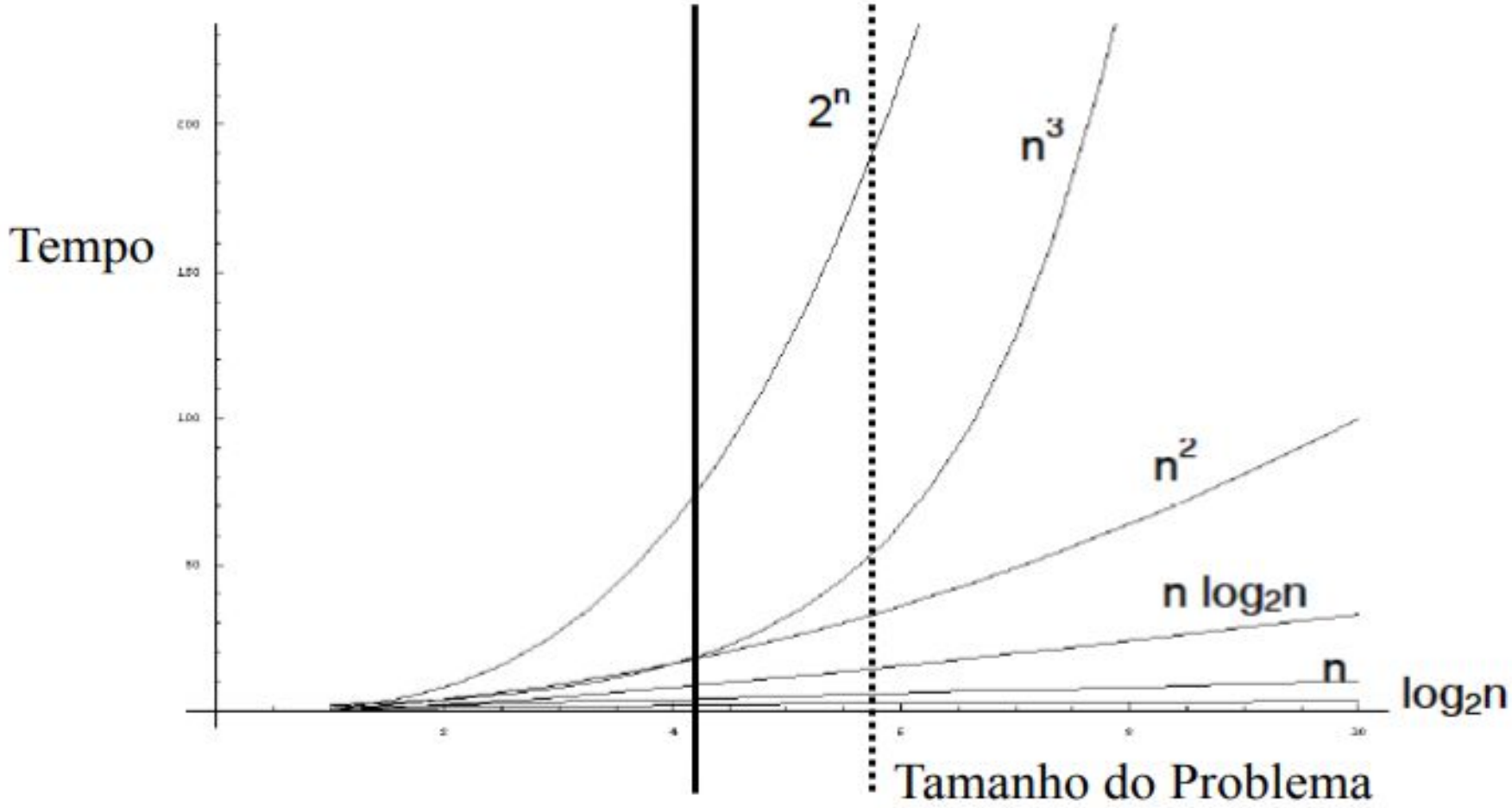
complexidade de tempo	máquina lenta	máquina rápida ($10x$)
$log_2 n$	x_0	x_0^{10}
n	x_1	$10x_1$
$n \log_2 n$	x_2	$10x_2$ (p/ x_2 grande)
n^2	x_3	$3.16x_3$
n^3	x_4	$2.15x_4$
2^n	x_5	$x_5 + 3.3$
3^n	x_6	$x_6 + 2.096$

Complexidade do Algoritmo x Tamanho máximo de problema resolvível

Toscani, L. V., Veloso, P. A. S., Complexidade de algoritmos: análise, projeto e métodos, 3 ed. Bookman, 2012
https://www.inf.ufrgs.br/~prestes/Courses/Complexity/aula1.pdf

Algoritmos

Complexidade x Tempo



Toscani, L. V., Veloso, P. A. S., Complexidade de algoritmos: análise, projeto e métodos, 3 ed. Bookman, 2012
https://www.inf.ufrgs.br/~prestes/Courses/Complexity/aula1.pdf

Análise de Algoritmos

- A complexidade de algoritmos é essencial em projetos considerando o tempo e recursos para para processamento e memória;
- Há situações que isso se torna inviável tecnologicamente ou financeiramente;
- O mesmo vale para a adoção de tecnologias que possam impactar nos resultados de tempo e recursos.

Análise de Algoritmos - Medidas

- Medir o tempo absoluto (em minutos, segundos, etc.) pode não ser interessante por depender da máquina.
- Usualmente conta-se o número de operações consideradas relevantes de serem realizadas pelo algoritmo e expressa-se esse número como uma função de n.
- Essas operações podem ser comparações, operações aritméticas, movimento de dados, etc.

Análise de Algoritmos - Pessimista/Médio e Melhor Caso

- **Pessimista:** estima-se o algoritmo pelo pior desempenho possível por entradas pré-estabelecidas;
- **Médio:** ou esperado é a complexidade média com todos os valores de entrada esperados, usando-se como peso a probabilidade de ocorrência de cada entrada;
- **Melhor:** situação em que pela entrada de dados estima-se o menor consumo de tempo e ou recursos;

Análise de Algoritmos - Pessimista/Médio e Melhor Caso

```c
#include <stdio.h>
int buscaSimples(int v[], int n, int x) {
   for (int i = 0; i < n; i++) {
       if (v[i] == x) {
           return i; // Encontrou o valor, retorna o índice
       }
   }
   return -1; // Valor não encontrado
}
int main() {
   int v[] = {3, 8, 12, 5, 9, 2};
   int n = sizeof(v);
   int x = 9;
   int pos = buscaSimples(v, n, x);
   if (pos != -1)
       printf("Valor encontrado na posição %d\n", pos);
   else
       printf("Valor não encontrado\n");
   return 0;
}
```

Análise de Algoritmos - Pessimista/Médio e Melhor Caso

```c
#include <stdio.h>
int buscaSimples(int v[], int n, int x) {
   for (int i = 0; i < n; i++) {
       if (v[i] == x) {
           return i; // Encontrou o valor, retorna o índice
       }
   }
   return -1; // Valor não encontrado
}
```

Linha de código	Tipo de operação	Quantas vezes ocorre?
`int i = 0`	Atribuição inicial	1 vez
`i < n`	Comparação da condição do loop	(n + 1) vezes
`i++`	Incremento	(n) vezes
`v[i] == x`	Comparação dentro do laço	Até (n) vezes
`return i` (caso encontrado)	Interrupção do algoritmo (pode ocorrer antes)	depende do caso

Análise de Algoritmos - Pessimista/Médio e Melhor Caso

Linha de código	Quantas vezes ocorre?	Quantidade
`int i = 0`	1 vez	1
`i < n`	(n + 1) vezes	(n+1)
`i++`	(n) vezes	(n)
`v[i] == x`	Até (n) vezes	(n)
`return i` (caso encontrado)	depende do caso	1

$$Total = 1 + (n + 1) + 2 \cdot n + 1 = 3 \cdot n + 3$$

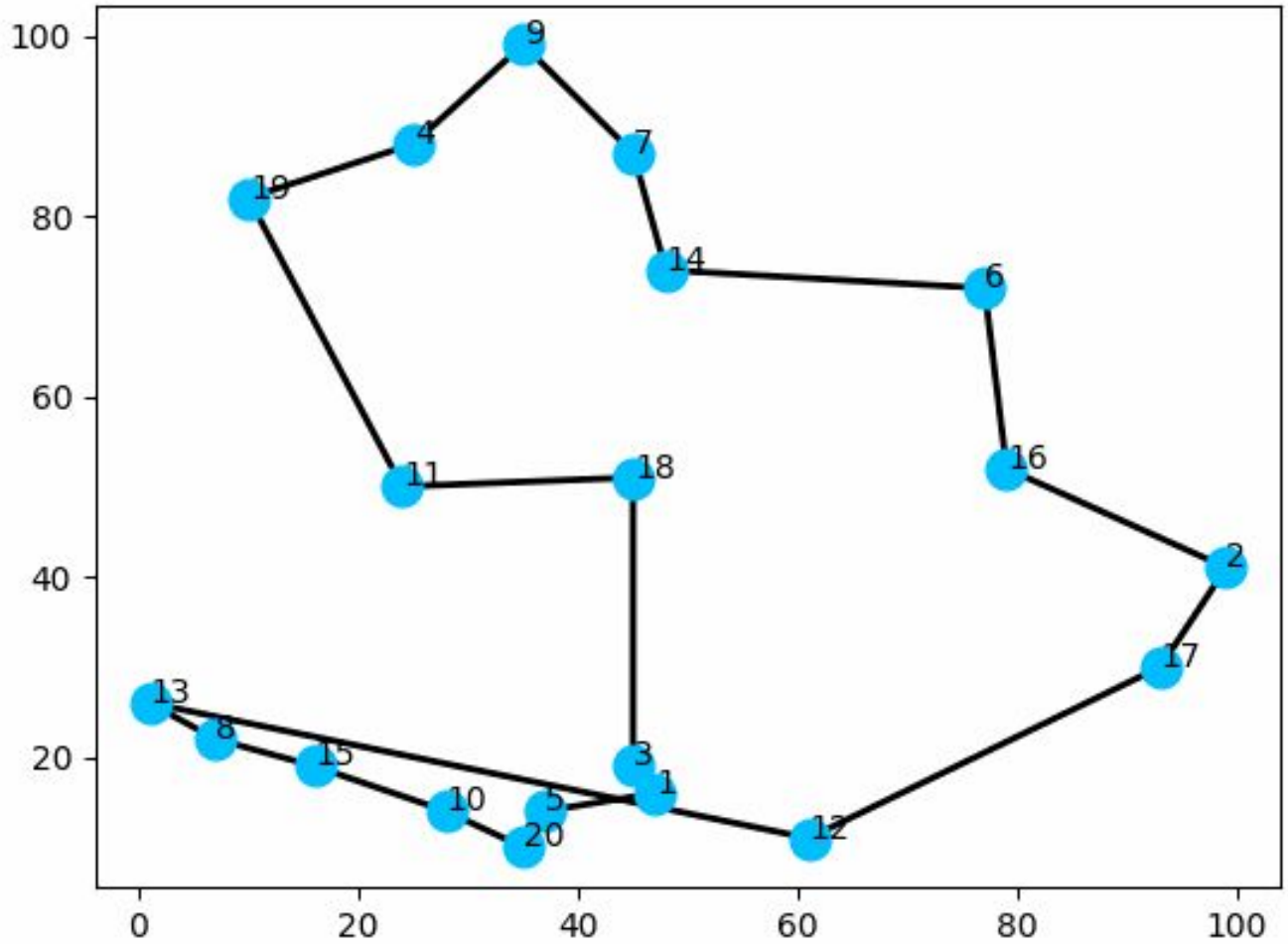
Análise de Algoritmos - Pessimista/Médio e Melhor Caso

Linha de código	Quantas vezes ocorre?	Quantidade
`int i = 0`	1 vez	1
`i < n`	(n + 1) vezes	(n+1)
`i++`	(n) vezes	(n)
`v[i] == x`	Até (n) vezes	(n)
`return i` (caso encontrado)	depende do caso	1

Caso	Função de complexidade	Ordem de crescimento
Melhor caso	(T(n) = O(1))	Constante
Caso médio	(T(n) = O(n))	Linear
Pior caso	(T(n) = O(n))	Linear

Análise de Algoritmos - Exemplos de Aplicações

Caixeiro viajante (Traveling Salesman Person - TSP)

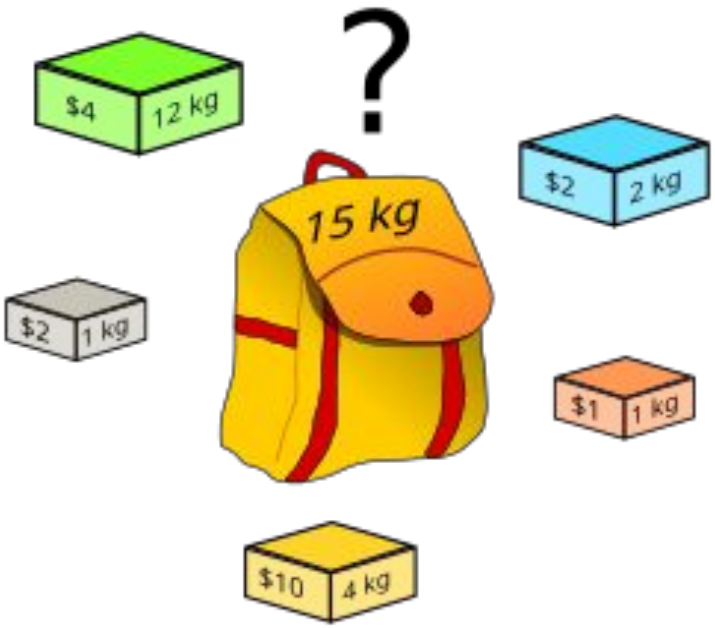


https://dobslit.com/resolvendo-o-problema-do-caixeiro-viajante-via-computacao-quantica-adiabatica/

Análise de Algoritmos - Exemplos de Aplicações

Caixeiro viajante (Traveling Salesman Person - TSP)

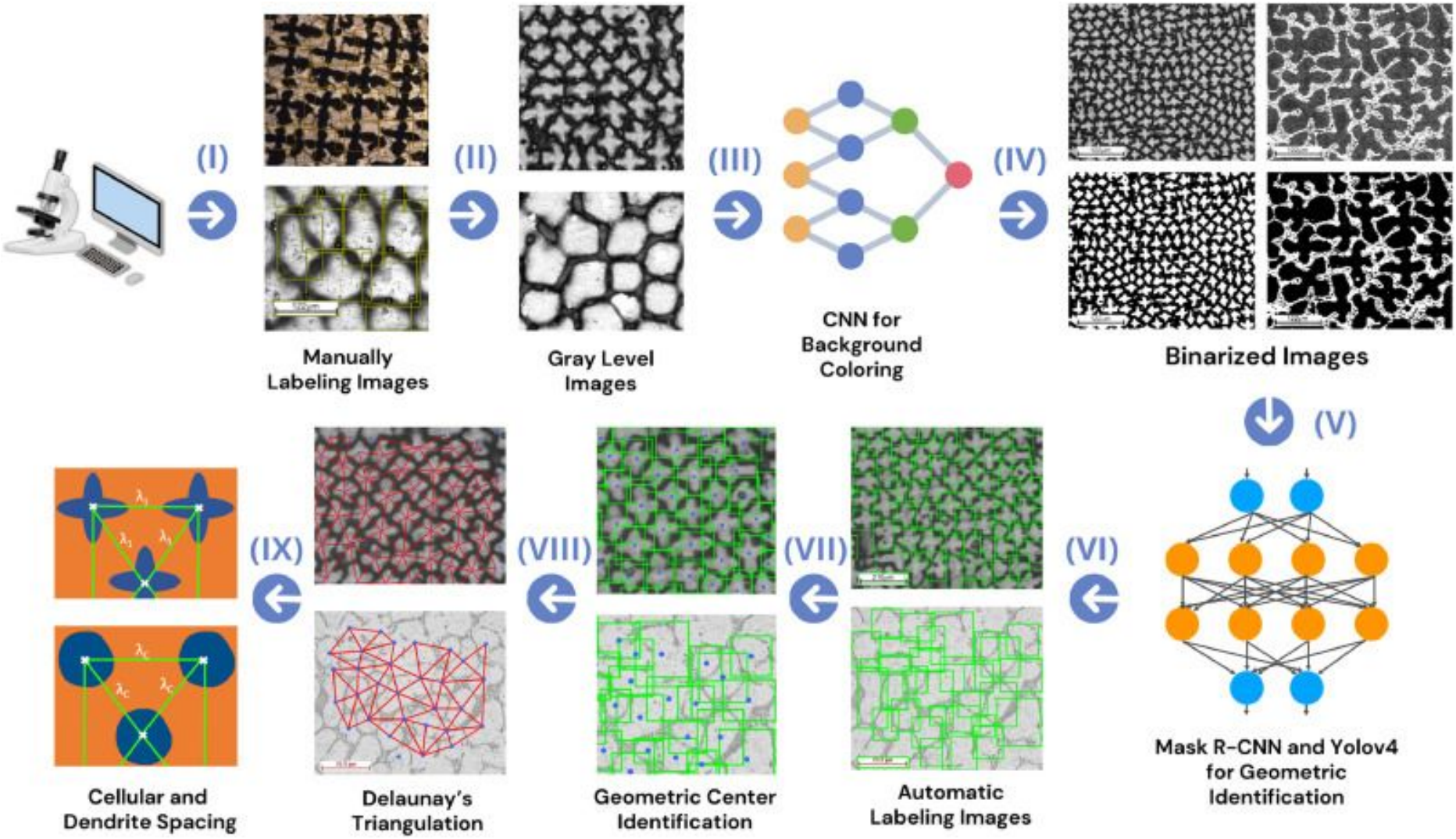
O **problema da mochila** (em inglês, *Knapsack problem*) é um problema de optimização combinatória. O nome dá-se devido ao modelo de uma situação em que é necessário preencher uma mochila com objetos de diferentes pesos e valores. O objetivo é que se preencha a mochila com o maior valor possível, não ultrapassando o peso máximo.



https://pt.wikipedia.org/wiki/Problema_da_mochila

Análise de Algoritmos - Exemplos de Aplicações

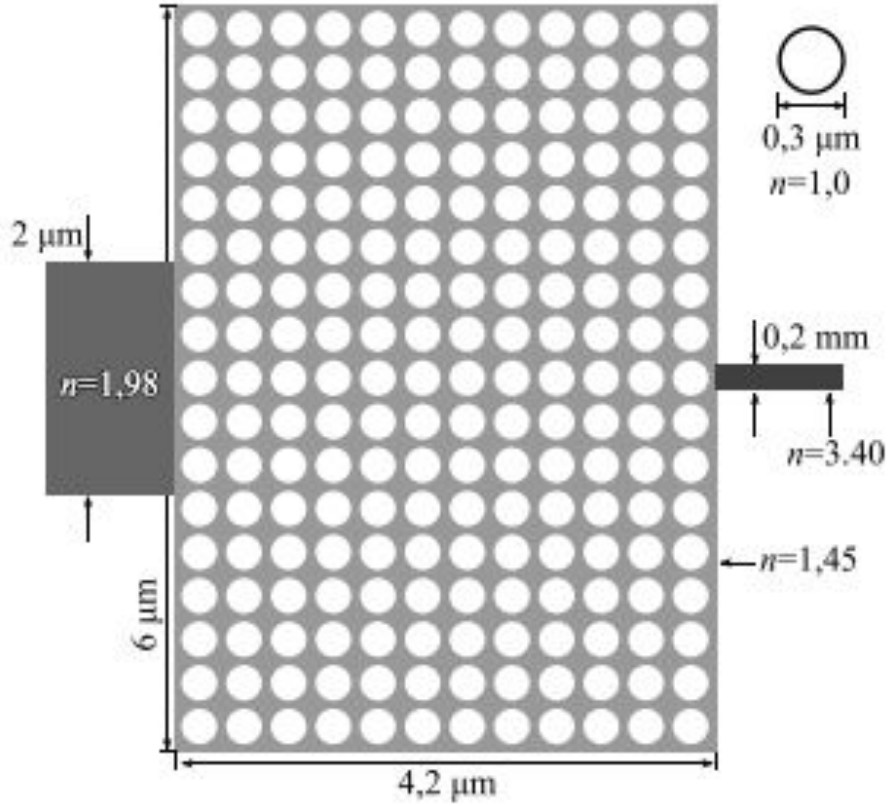
Aprendizado de Máquina



DA SILVA, Guilherme Marim et al. Automated Recognition and Measurement of Cellular and Dendritic Microstructures in Alloys Via Machine Learning. Available at SSRN 5334590.

Análise de Algoritmos - Exemplos de Aplicações

IA e Análise Combinatória (n=3.5 ou 1.0)



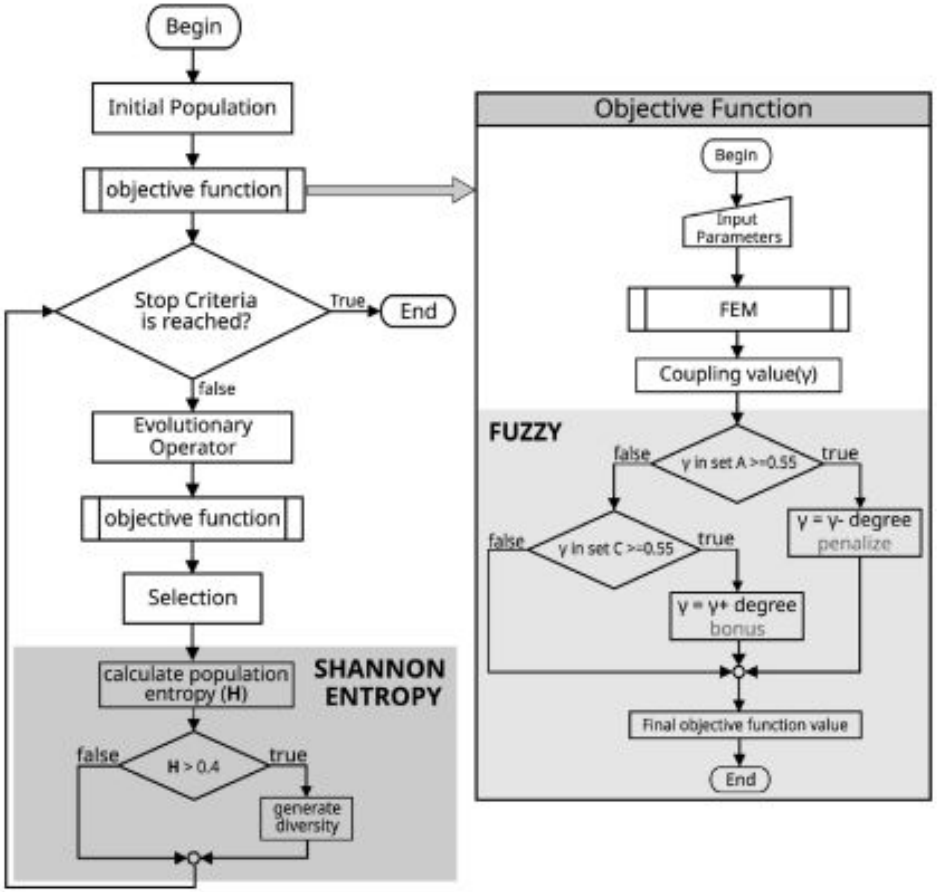


Fig. 3. Flowchart of the algorithms used.

JUNIOR, Wilton Moreira Ferraz; GONÇALVES, Marcos Sergio; DA SILVA SANTOS, Carlos Henrique. Evolutionary strategy combined to Fuzzy Logic and Shannon's Entropy to maximize the optimization of optical coupler design. SoftwareX, v. 27, p. 101835, 2024.

Análise de Algoritmos - Exemplos de Aplicações

IA e Análise Combinatória (n=3.5 ou 1.0)

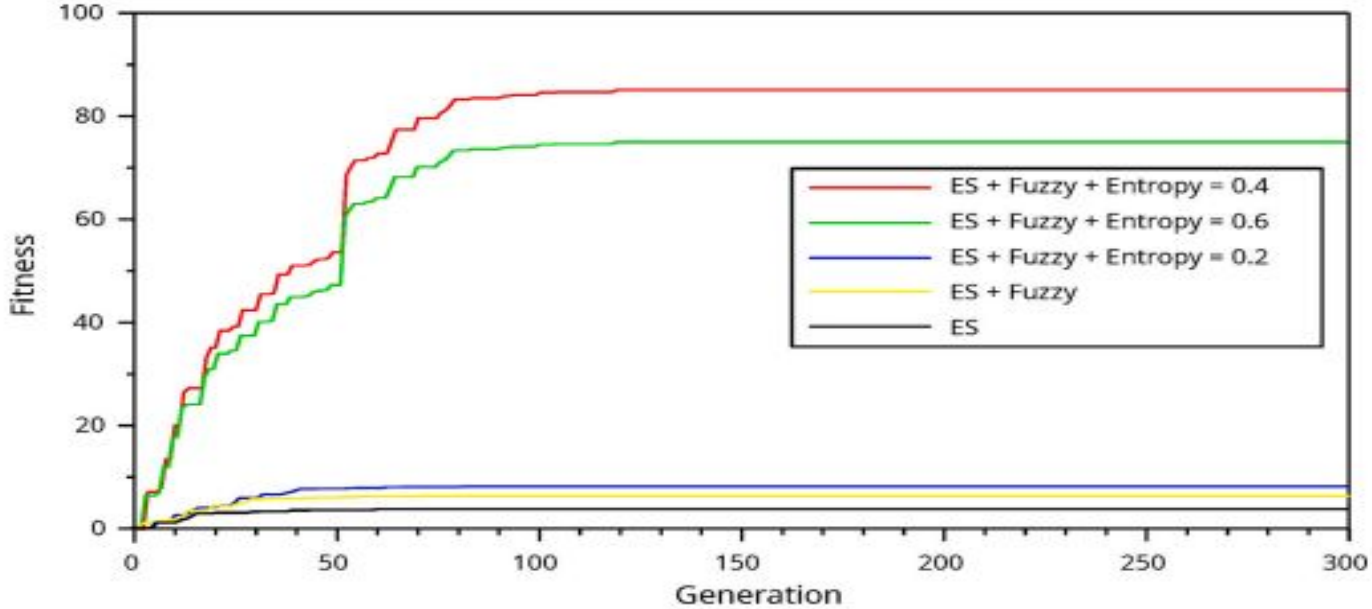


Fig. 6. Comparison chart of the results obtained in the optimizations.

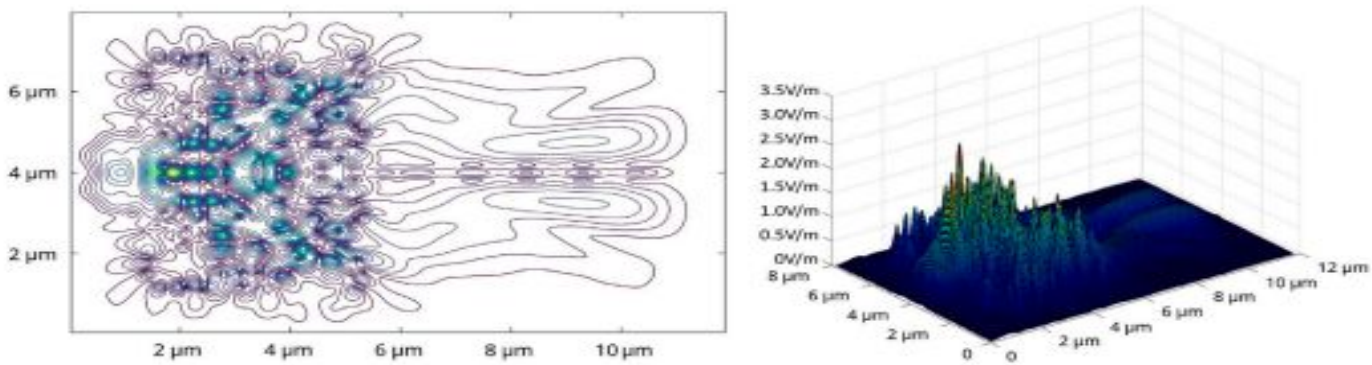


Fig. 7. Two-dimensional visualization of the electric field distribution in the micro-to-nano coupler of optimizations using only ES.

JUNIOR, Wilton Moreira Ferraz; GONÇALVES, Marcos Sergio; DA SILVA SANTOS, Carlos Henrique. Evolutionary strategy combined to Fuzzy Logic and Shannon's Entropy to maximize the optimization of optical coupler design. SoftwareX, v. 27, p. 101835, 2024.

Como calculamos a complexidade (tempo e,ou, consumo) desses algoritmos?

Vamos rever/entender Somatório e Produtório

- Somatório é uma operação matemática que representa a adição de uma sequência de termos, usando o símbolo grego sigma (Σ) para expressar essa soma de forma concisa.
- Ele é muito utilizado para simplificar fórmulas, especialmente em áreas como estatística, economia e análise combinatória

$$\sum_{i=1}^{4} i = 1+2+3+4 = 10$$

https://algol.dev/somatorios-o-que-sao/

Vamos rever/entender Somatório e Produtório

$$\sum_{i=1}^{4} i = 1+2+3+4 = 10$$

$$\sum_{i=1}^{4} 2 = 2+2+2+2 = 8$$

$$\sum_{i=1}^{4} 2i - 1 = (2\text{x}\mathbf{1}-1)+(2\text{x}\mathbf{2}-1)+(2\text{x}\mathbf{3}-1)+(2\text{x}\mathbf{4}-1) = 1+3+5+7 = 16$$

https://algol.dev/somatorios-o-que-sao/

Vamos rever/entender Somatório e Produtório

$$\sum_{i=2}^{5} i^2 = ?$$

$$\sum_{i=0}^{1} 10^i = ?$$

$$\sum_{i=5}^{10} 2 + i = ?$$

https://algol.dev/somatorios-o-que-sao/

Vamos rever/entender Somatório Propriedades

(i) $$\sum_{i\in I}(a_i + b_i) = \sum_{i\in I} a_i + \sum_{i\in I} b_i$$

(ii) $$\sum_{i\in I}(a_i - b_i) = \sum_{i\in I} a_i - \sum_{i\in I} b_i$$

(iii) Se $I \cap J = \varnothing$, então

$$\sum_{i\in I\cup J} a_i = \sum_{i\in I} a_i + \sum_{i\in J} a_i.$$

(iv) $$\sum_{i\in I \text{ e } j\in J} a_i b_j = \sum_{i\in I} a_i . \sum_{j\in J} b_j.$$

(v) $$\sum_{i\in I} 1 = |I|.$$

https://olimpedia.fandom.com/pt-br/wiki/Somat%C3%B3rios_e_Produt%C3%B3rios#Produt%C3%B3rios

Vamos rever/entender Somatório e Produtório

$$\prod_{i=m}^{n} a_i = a_m a_{m+1} a_{m+2} \cdots a_{n-1} a_n .$$

$$\prod_{i=1}^{5} i = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120$$

$$\prod_{i=0}^{3} x_i = x_0 \cdot x_1 \cdot x_2 \cdot x_3$$

https://olimpedia.fandom.com/pt-br/wiki/Somat%C3%B3rios_e_Produt%C3%B3rios#Produt%C3%B3rios

Exemplo 1

```
for (int i = 1; i <= n; i++) {
    // Operações de tempo constante (custo c)
}
```

O custo total é a soma do custo constante (c) para cada iteração, de $i = 1$ a n.

$$T(n) = \sum_{i=1}^{n} c = c \cdot n$$

Exemplo 2 - Como fica ?

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        // Operações de tempo constante (custo c)
    }
}
```

$$T(n) = \sum_{j=1}^{n} c = c \cdot n$$

for interno

Exemplo 2 - Como fica ?

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        // Operações de tempo constante (custo c)
    }
}
```

$$T(n) = \sum_{i=1}^{n} \sum_{j=1}^{i} c$$

$$T(n) = \sum_{i=1}^{n} (c \cdot i) = c \sum_{i=1}^{n} i$$

Usando a fórmula do somatório de uma progressão aritmética

$$\left(\sum_{i=1}^{n} i = \frac{n(n+1)}{2}\right):$$

$$T(n) = c \cdot \frac{n(n+1)}{2} = \frac{c}{2}(n^2 + n)$$

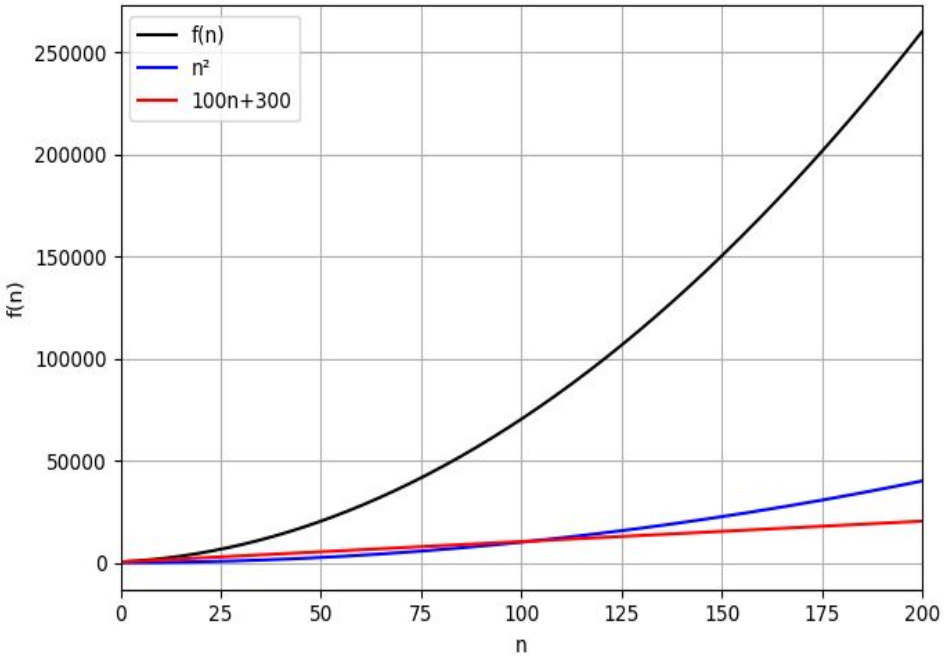
A complexidade assintótica é dominada pelo termo de maior grau **(n^2)**

Notações Assintóticas O, Θ e Ω

Notação Assintótica

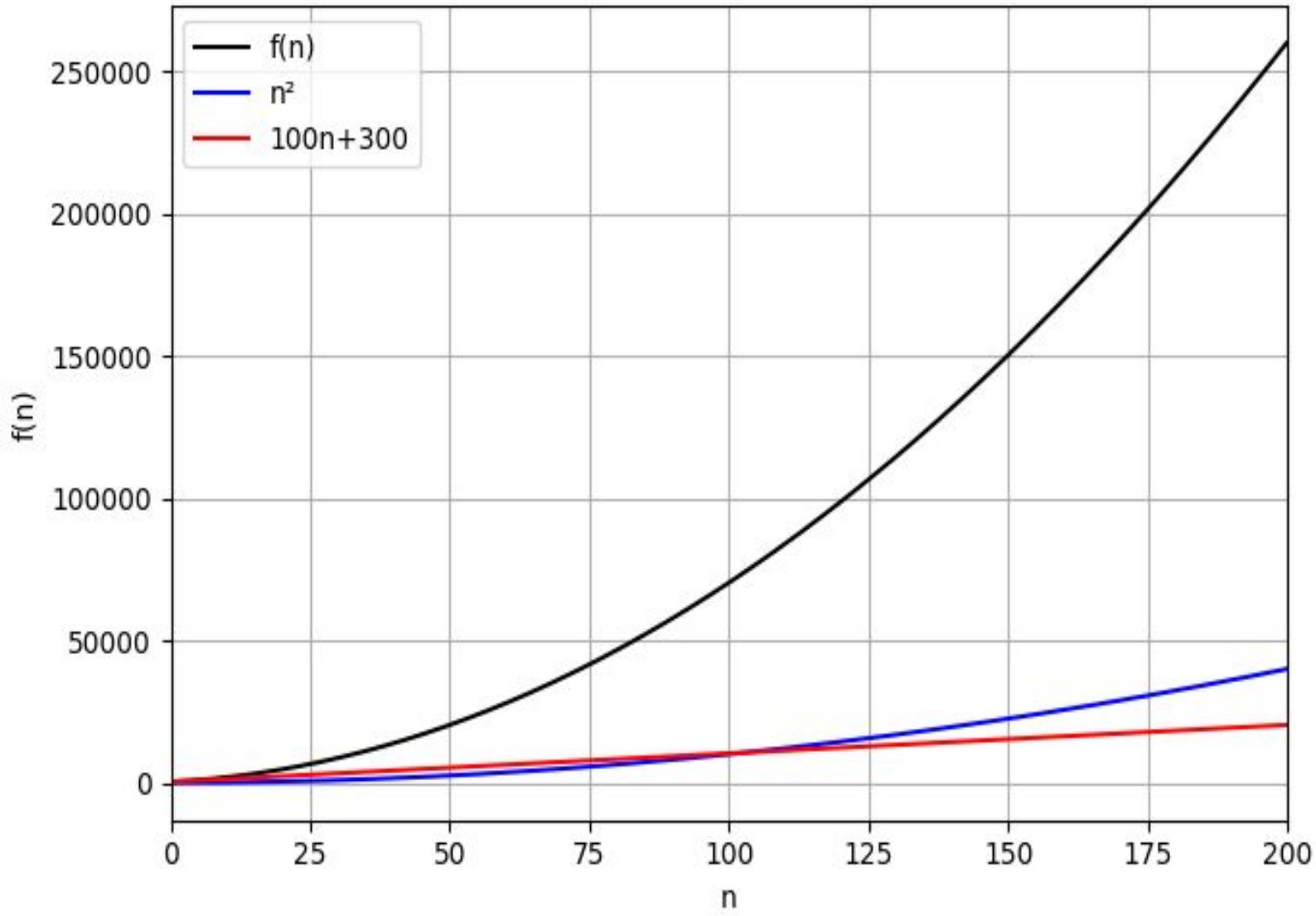
Quando descartamos os coeficientes constantes e os termos menos significativos, usamos notação assintótica. Vamos estudar suas três formas: notação O, Θ e Ω.

Por exemplo, suponha que um algoritmo, sendo executado com uma entrada de tamanho **n**, leve $\mathbf{6 \cdot n^2 + 100 \cdot n + 300}$ instruções de máquina.



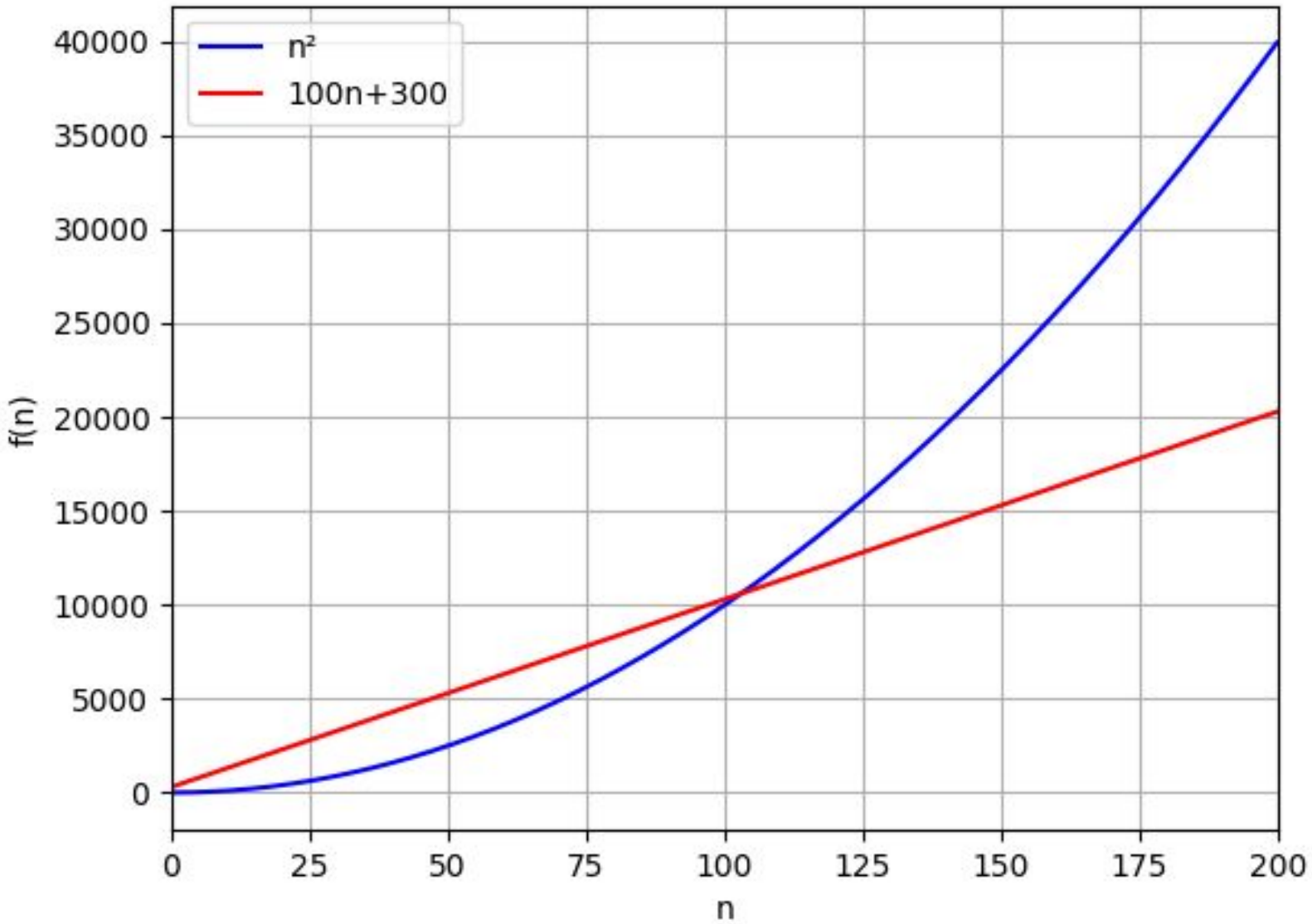
https://pt.khanacademy.org/computing/computer-science/algorithms/asymptotic-notation/a/asymptotic-notation

Notação Assintótica



https://pt.khanacademy.org/computing/computer-science/algorithms/asymptotic-notation/a/asymptotic-notation

Notação Assintótica



https://pt.khanacademy.org/computing/computer-science/algorithms/asymptotic-notation/a/asymptotic-notation

Notação Assintótica

```
import matplotlib.pyplot as plt
import numpy as np
n = 1000
minn = 0
maxn = 200
step = (maxn-minn)/n
n = np.arange(minn,maxn,step)
fn = 6 * n **2 + 100 * n + 300
fn_parcial_potencia = n ** 2
fn_parcial_n = 100 * n + 300
plt.plot(n,fn, 'k', label='f(n)')
plt.plot(n,fn_parcial_potencia, 'b', label='n²')
plt.plot(n,fn_parcial_n, 'r', label='100n+300')
plt.grid(1)
plt.legend()
plt.xlabel('n')
plt.ylabel('f(n)')
plt.xlim([minn, maxn])
plt.show()
```

Notação Assintótica

- **Notação O:** É a mais comum e indica o limite superior assintótico de um algoritmo, ou seja, o pior caso. Se um algoritmo é ($O(n^2)$), significa que seu desempenho cresce no máximo no ritmo de n^2 para entradas grandes;
- **Notação Ω:** Indica o limite inferior assintótico, o melhor caso de desempenho possível de um algoritmo. Por exemplo, um algoritmo pode ser ($\Omega(n)$).
- **Notação θ:** Representa o limite exato (ou "estreito"). Se um algoritmo é ($\theta(n)$), significa que seu crescimento é limitado superior e inferiormente pela mesma função, sendo aproximadamente igual ao longo do tempo.

Notação Assintótica - $f(n)=6 \cdot n^2 + 100 \cdot n + 300$

- **Notação O:** A notação Big-O representa o limite superior do tempo de execução de um algoritmo, ou seja, o pior caso. Para a função $f(n)=6 \cdot n^2 + 100 \cdot n + 300$, o termo de maior ordem é n^2. Portanto, **f(n)** é $O(n^2)$. Isso significa que o tempo de execução do algoritmo não crescerá mais rápido do que uma função quadrática.
- **Notação Ω:** A notação **Ω** representa o limite inferior do tempo de execução de um algoritmo, ou seja, o melhor caso. Para a função $f(n)=6 \cdot n^2 + 100 \cdot n + 300$, o termo de maior ordem é n^2. Portanto, **f(n)**é $\Omega(n^2)$ Isso significa que o tempo de execução do algoritmo não será mais lento do que uma função quadrática.

Notação Assintótica - $f(n)=6 \cdot n^2 + 100 \cdot n + 300$

- **Notação θ:** A notação **θ** representa um limite assintoticamente apertado, indicando que o limite superior e o limite inferior são os mesmos. Para a função $\mathbf{f(n)=6 \cdot n^2 + 100 \cdot n + 300}$, como os limites superior e inferior são ambos $\mathbf{n^2}$, a função é $\boldsymbol{\theta(n^2)}$. Isso significa que o tempo de execução do algoritmo cresce na mesma proporção que uma função quadrática.

Notação Assintótica - $f(n)=5n^3+2n$

- **Notação O (Limite Superior):** A função é $\mathbf{O(n^3)}$ e ocorre devido ao termo $\mathbf{(5n^3)}$ domina o crescimento. A notação Big O indica que o crescimento de **f(n)** é no máximo na mesma ordem de $\mathbf{n^3}$. É possível encontrar uma constante **c** (por exemplo, **c=6**) e um n_0 (por exemplo, $n_0=1$) tal que $5n^3+2n \leq 6n^3$ para todo ($n \geq n_0$).
- Notação **Ω** (Limite Inferior): A função é $\mathbf{\Omega(n^3)}$. A notação **Ω** indica que o crescimento de **f(n)** é no mínimo na mesma ordem de $\mathbf{n^3}$. É possível encontrar uma constante **c**, por exemplo, **c=5** e um n_0 (por exemplo, $n_0=1$) tal que $5n^3 +2 \geq 5n^3$ para todo $n \geq n_0$.

Notação Assintótica - $f(n)=5n^3+2n$

- Notação **θ** (Limite Firme): A função é $\boldsymbol{\theta(n^3)}$. Como a função é tanto $\mathbf{O(n^3)}$ quanto $\boldsymbol{\Omega(n^3)}$, o crescimento de **f(n)** está "limitado firmemente" por $\mathbf{n^3}$. A notação **θ** é a mais precisa para descrever o comportamento assintótico.

Estratégia Geral para Escolher c e n_0

1) Identifique o termo dominante de f(n) (maior grau polinomial) — testamos g(n) a partir dele.
2) Reescreva a desigualdade $f(n) \leq c \cdot g(n)$ isolando termos não dominantes (ex.: divide tudo por g(n)).
3) Use desigualdades simples (por exemplo, para $n \geq 1$, $n \leq n^k$ quando $k \geq 1$) para dominar termos menores.
4) Escolha c como soma das constantes resultantes; escolha n_0 para garantir as desigualdades usadas.

Estratégia Geral para Escolher c e n_0

$f(n)=5n^3+2n$

1) Queremos encontrar **$c>0$** e **n_0** tal que **$5n^3 + 2n \leq c \cdot n^3$** para todo **$n \geq n_0$**.
2) Divida por **n^3 -> $(5n^3 + 2n)/ n^3 = 5 + 2/n^2 \leq c$**.
3) Como **$2/n^2 \leq 2$** para **$n \geq 1$**? Então para **$n \geq 1$**, **$2/n^2 \leq 2$**.
4) Mas queremos um limite que vá para **0** com **n** grande; note que **$2/n^2 \leq 2$** quando **$n \geq 1$**, então **$5 + 2/n^2 \leq 7$** para **$n \geq 1$**.
5) Portanto podemos escolher **$c=7$** e **$n_0=1$**. Assim, **$5n^3+2n \leq 7n^3$** para todo **$n \geq 1$**.

Dedução para se chegar nos valores para $f(n)=5n^3+2n$

Queremos comparar o termo $\frac{2}{n^2}$ para $n \geq 1$.

1. Para $n \geq 1$ temos $n^2 \geq 1$.
2. Como $n^2 \geq 1$, ao tomar recíproco (uma função decrescente em $x > 0$) obtemos $\frac{1}{n^2} \leq 1$.
3. Multiplicando essa desigualdade por 2 (número positivo) dá $\frac{2}{n^2} \leq 2$.

Além disso, por $n^2 > 0$ sempre, $\frac{2}{n^2} \geq 0$. Portanto, para $n \geq 1$ temos a cadeia

$$0 \leq \frac{2}{n^2} \leq 2.$$

Somando 5 em todos os termos:

$$5 \leq 5 + \frac{2}{n^2} \leq 7 \qquad \text{para todo } n \geq 1.$$

Isso mostra imediatamente que $5n^3 + 2n = n^3\left(5 + \frac{2}{n^2}\right) \leq 7n^3$ para $n \geq 1$. Logo podemos escolher $c = 7$ e $n_0 = 1$.

Dedução para se chegar nos valores para $f(n)=5n^3+2n$

Porque a expressão original

$$\frac{5n^3+2n}{n^3} = 5 + \frac{2}{n^2}$$

contém exatamente esse termo $5 + \frac{2}{n^2}$.

Assim, mostramos que:

$$5 + \frac{2}{n^2} \leq 7 \quad \text{para todo } n \geq 1$$

Multiplicando ambos os lados por n^3 (positivo, pois $n > 0$):

$$5n^3 + 2n \leq 7n^3$$

Exemplo 3 - $f(n)=3n^2 + 5n + 7$

Queremos encontrar uma notação assintótica simples.

$$f(n) = 3n^2 + 5n + 7$$

Dividir pelo termo dominante n^2

$$\frac{f(n)}{n^2} = 3 + \frac{5}{n} + \frac{7}{n^2}$$

Quando $n \to \infty$

$$\frac{5}{n} \to 0 \quad \text{e} \quad \frac{7}{n^2} \to 0$$

O limite é uma constante positiva $\lim_{n\to\infty} \frac{f(n)}{n^2} = 3$

$$f(n) = \Theta(n^2)$$

Isso significa que a função cresce proporcionalmente a $\mathbf{n^2}$

Exemplo 4 - f(n)=4·n·log(n)+20·n

Dividindo por n·log(n):

$$\frac{f(n)}{n \log n} = 4 + \frac{20}{\log n}$$

Quando $n \to \infty$, $\frac{20}{\log n} \to 0$

Então

$$f(n) = \Theta(n \log n)$$

Comparando crescimento x^3 e x^2

Queremos comparar $f(n)=n^2$ e $g(n)=n^3$

$$\frac{f(n)}{g(n)} = \frac{n^2}{n^3} = \frac{1}{n} \to 0$$

Isso mostra que f(n) **cresce muito mais devagar** que g(n).

Logo,

$$f(n) = o(g(n))$$

(leia “f é pequeno o de g”)

*Interpretação intuitiva: n^2 é **insignificante** comparado a n^3 quando n é muito grande*

Comparando crescimento linear e quadrático

Compare f(n)=10n+5 com g(n)=n^2

$$\frac{f(n)}{g(n)} = \frac{10n+5}{n^2} = \frac{10}{n} + \frac{5}{n^2} \to 0$$

Logo,

$$f(n) = o(n^2)$$

$$f(n) = O(n^2)$$

Cresce mais lentamente do que n^2

Obrigado!

Dúvidas?